Technická fakulta

Systém pro správu skladu elektronických součástek

Předmět:	Projektování IS
Kód předmětu:	EIT21E
Autoři:	Tomáš Simandl
	Tomáš Szetei
	Lukáš Kvapil
Datum:	27. dubna 2026

Obsah

1 Formulace problému **3**

1.1 Účel . . . 3

1.2 Specifikace . . . 3

2 Datový slovník **4**

2.1 Soucastka . . . 4

2.2 Projekt . . . 4

2.3 SkladovyPohyb . . . 5

2.4 PrijemPohyb . . . 5

2.5 VydejPohyb . . . 5

2.6 PresunutiPohyb . . . 6

2.7 Sklad . . . 6

2.8 Lokace . . . 6

2.9 Uzivatel . . . 6

3 Objektový model **7**

4 Dynamický model **9**

4.1 Stavový diagram třídy SkladovyPohyb . . . 9

4.2 Stavový diagram třídy Soucastka . . . 10

5 Model interakcí **11**

5.1 Use Case diagram . . . 11

5.2 Scénáře . . . 12

5.2.1 Scénář: Příjem na sklad (PrijemPohyb) . . . 12

5.2.2 Scénář: Výdej ze skladu (VydejPohyb) . . . 12

5.2.3 Scénář: Přesun součástky (PresunutiPohyb) . . . 12

5.2.4 Scénář: Přiřazení součástky k projektu . . . 13

5.2.5 Scénář: Správa součástek . . . 13

5.3 Sekvenční diagram – Vytvoření pohybu . . . 13

5.4 Diagram aktivit – Vytvoření pohybu . . . 15

6 Závěr **17**

A Zdrojové kódy PlantUML diagramů **18**
A.1 Use Case diagram 18
A.2 Class diagram 19
A.3 Stavový diagram – SkladovyPohyb 21
A.4 Stavový diagram – Soucastka 22
A.5 Sekvenční diagram – Vytvoření pohybu 23
A.6 Diagram aktivit – Vytvoření pohybu 24

1. Formulace problému

1.1 Účel

Cílem tohoto projektu je teoreticky navrhnout informační systém pro správu skladových součástek. Systém má umožňovat evidenci součástek, jejich kategorií, dodavatelů a skladových lokací. Dále má podporovat sledování pohybů součástek ve skladu, jako je naskladnění, vyskladnění nebo přesun mezi jednotlivými lokacemi.

Součástí systému je také možnost přiřazovat součástky k projektům a evidovat jejich množství potřebné pro daný projekt.

1.2 Specifikace

Návrh systému je znázorněn pomocí UML diagramů, které popisují jednotlivé třídy, jejich atributy, vztahy mezi nimi a chování systému při klíčových operacích. Systém zahrnuje objektový model (class diagram), dynamický model (stavové diagramy) a model interakcí (use case diagram, scénáře, sekvenční a aktivity diagram).

2. Datový slovník

2.1 Soucastka

Skladová položka evidovaná v systému WMS s aktuálním množstvím na skladě. Atribut rezervovaneMnozstvi sleduje, kolik kusů je aktuálně blokováno pro projekty – součástka tak může být Dostupna i s částečnou rezervací zároveň. Součástka může nabývat stavů: Dostupna (k dispozici, případně s rezervovanou částí), Vydana (fyzicky vydána ze skladu) nebo Odepsana (trvale vyřazena z evidence).

Atributy:

- id: int
- nazev: String
- mnozstvi: int
- rezervovaneMnozstvi: int
- stav: String (Dostupna | Vydana | Odepsana)

Metody:

- zvysMnozstvi(pocet: int)
- snizMnozstvi(pocet: int)
- rezervovat(pocet: int)
- zrusitRezervaci(pocet: int)
- overDostupnost(): boolean

2.2 Projekt

Projekt, ke kterému jsou přiřazeny součástky pro sledování spotřeby.

Atributy:

- id: int
- nazev: String

Metody:

- priradSoucastku(Soucastka)
- odeberSoucastku(Soucastka)

2.3 SkladovyPohyb

Abstraktní třída reprezentující pohyb součástek – příjem, výdej nebo přesun. Každý pohyb prochází životním cyklem: vytvoření, validace dat a provedení pohybu.

Atributy:

- id: int
- datum: Date
- stav: String (Dostupna | Vydana | Odepsana)
- typ: String (prijem | vydej | presun)
- mnozstvi: int

Metody:

- {abstract} vytvoritPohyb()
- overitData()
- provestPohyb()

2.4 PrijemPohyb

Konkrétní pohyb typu příjem – naskladnění součástek od dodavatele. Dědí ze SkladovyPohyb.

Atributy:

- dodavatel: String – název dodavatele součástek
- dědí ze SkladovyPohyb: id, datum, stav, typ, mnozstvi

Metody:

- vytvoritPohyb()

2.5 VydejPohyb

Konkrétní pohyb typu výdej – vyskladnění součástek ze skladu. Dědí ze SkladovyPohyb. Atribut duvod popisuje účel výdeje, např. použití v projektu, reklamace, interní spotřeba nebo oprava.

Atributy:

- duvod: String – účel výdeje (projekt, reklamace, interní spotřeba, oprava)
- dědí ze SkladovyPohyb: id, datum, stav, typ, mnozstvi

Metody:

- vytvoritPohyb()

2.6 PresunutiPohyb

Konkrétní pohyb typu přesun – přemístnění součástek mezi lokacemi ve skladu. Dědí ze SkladovyPohyb.

Atributy:

- zdrojLokaceId: int – ID zdrojové lokace
- cilLokaceId: int – ID cílové lokace
- dědí ze SkladovyPohyb: id, datum, stav, typ, mnozstvi

Metody:

- vytvoritPohyb()
- zmenLokaci()

2.7 Sklad

Fyzický sklad rozdělený na lokace, ve kterých jsou uloženy součástky.

Atributy:

- id: int
- nazev: String

Metody: najdiSoucastku(id: int): Soucastka

2.8 Lokace

Konkrétní místo v skladu identifikované označením (např. A-01-03).

Atributy:

- id: int
- oznaceni: String

2.9 Uzivatel

Osoba spravující součástky, pohyby, projekty a sklady v systému.

Atributy:

- id: int
- jmeno: String
- email: String

3. Objektový model

Diagram zachycuje strukturu systému WMS a vztahy mezi jeho hlavními třídami. Součástky jsou přiřazovány k projektům a fyzicky umístěny v lokacích skladu. Třída Uzivatel reprezentuje uživatelskou roli v systému. Vytvoření pohybu provádí voláním SkladovyPohyb.vytvoritPohyb() a přiřazení součástky k projektu voláním Projekt.priradSoucastku().

Diagram využívá **generalizaci**: abstraktní třída SkladovyPohyb je základem pro tři konkrétní podtřídy PrijemPohyb (naskladnění), VydejPohyb (vyskladnění) a PresunutiPohyb (přesun mezi lokacemi), které dědí společné atributy a přidávají vlastní specifické atributy.

Třída SkladovyPohyb deklaruje abstraktní metodu vytvoritPohyb(), kterou každá podtřída implementuje po svém:

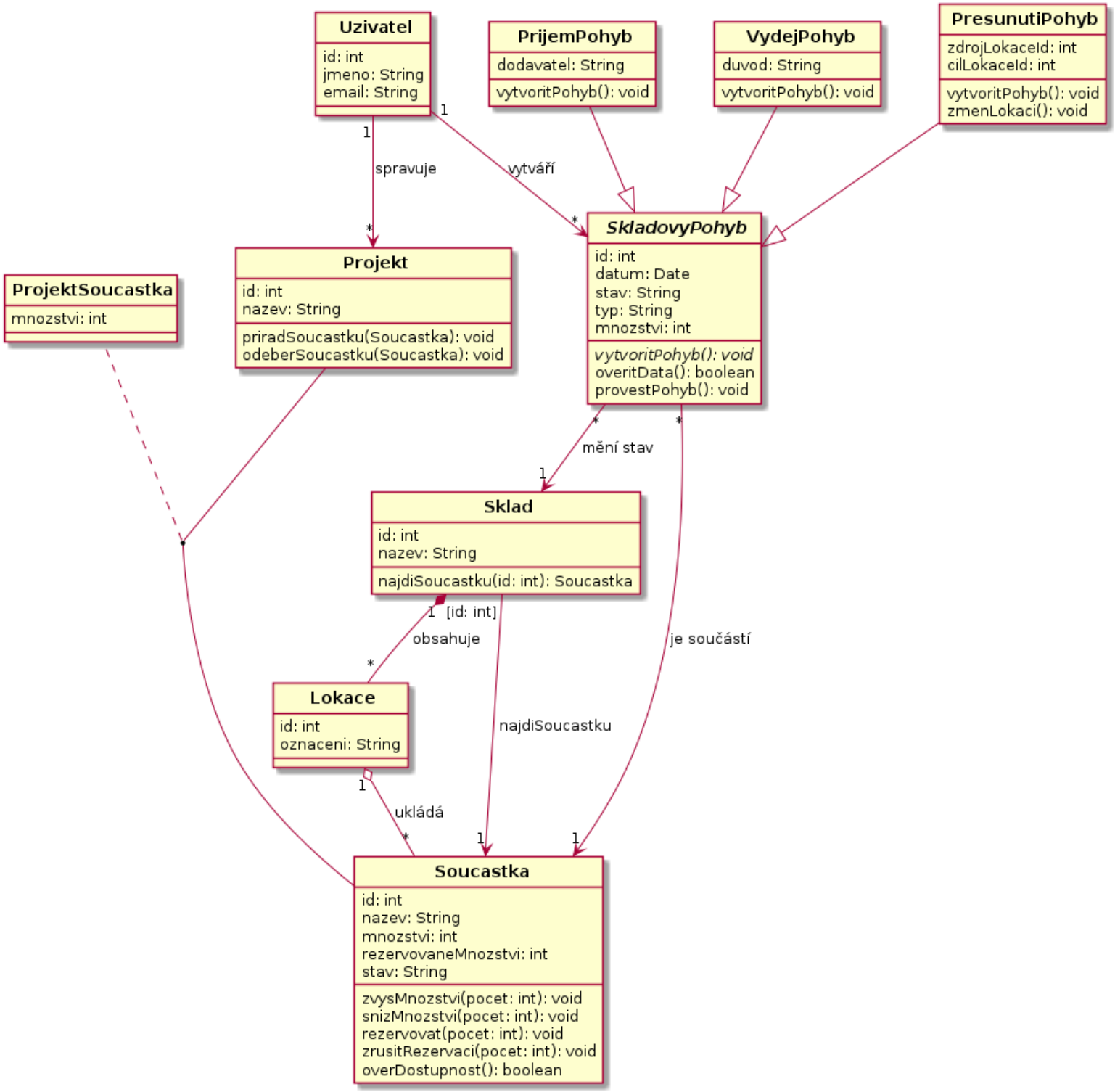
- PrijemPohyb.vytvoritPohyb() – vytvoří příjem a nastaví atribut dodavatel.
- VydejPohyb.vytvoritPohyb() – vytvoří výdej a nastaví atribut duvod.
- PresunutiPohyb.vytvoritPohyb() – vytvoří přesun a nastaví zdrojLokaceId a cilLokaceId.

Třída SkladovyPohyb zaznamenává každou změnu stavu zásob v systému. Konkrétně:

- PrijemPohyb – příjem součástek na sklad od dodavatele; zvyšuje množství dané součástky.
- VydejPohyb – výdej součástek ze skladu na projekt; snižuje množství dané součástky.
- PresunutiPohyb – přemístnění součástek mezi lokácemi ve skladu; mění přiřazenou lokaci součástky.

Každý pohyb se váže na konkrétní součástku (vazba je součástí, multiplicita $* \to 1$) a mění stav jednoho skladu (vazba mění stav, multiplicita $* \to 1$). Jeden uživatel může vytvořit libovolný počet pohybů (vazba vytváří, multiplicita $1 \to *$). Pohyb nemění fyzickou lokaci součástky – ta je pevně dána vazbou Lokace $\to$ Soucastka.

Vazba mezi Sklad a Soucastka je **kvalifikovaná** – sklad přistupuje ke konkrétní součástce prostřednictvím kvalifikátoru id: int.



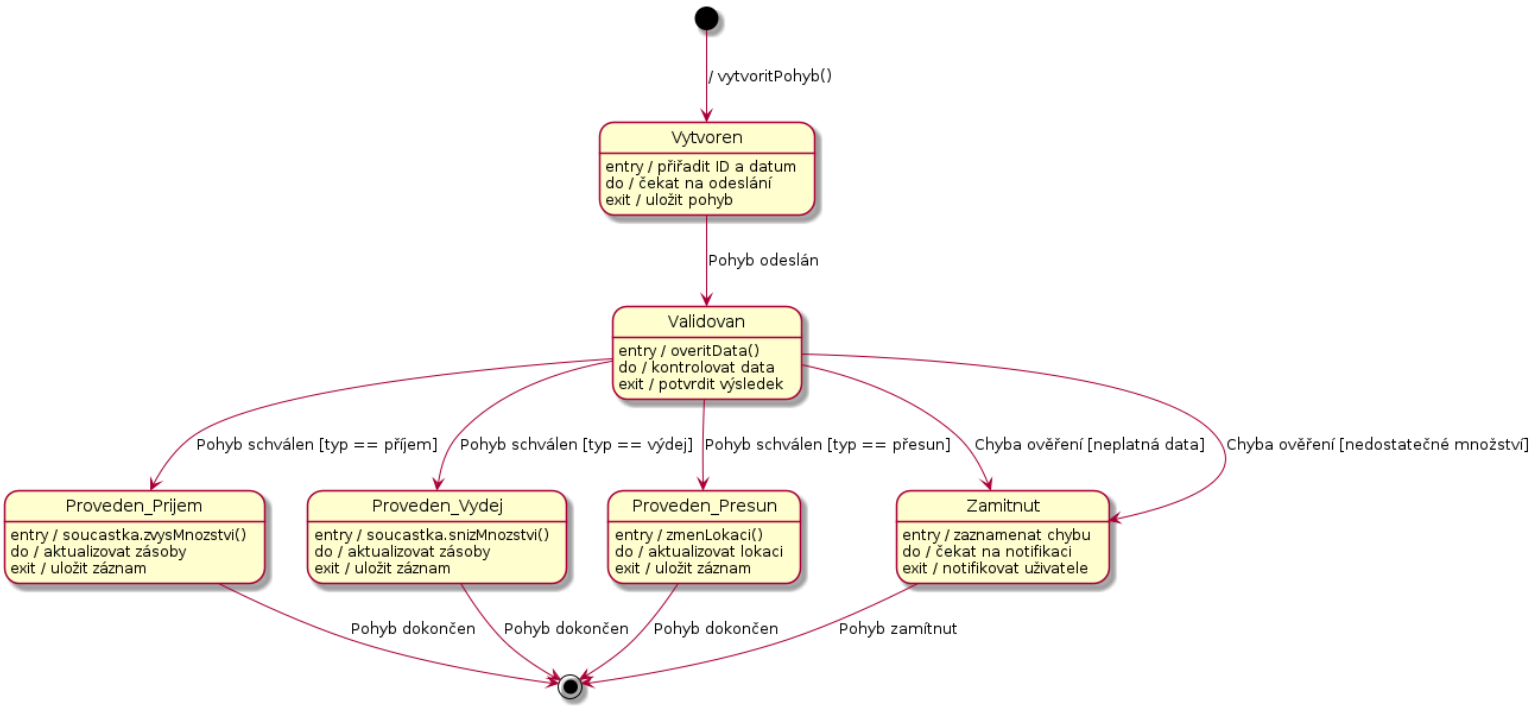
Obrázek 3.1: Class diagram

4. Dynamický model

4.1 Stavový diagram třídy SkladovyPohyb

Třída SkladovyPohyb reprezentuje obecný skladový pohyb – tedy jakoukoli změnu stavu zásob součástek v systému. Každý pohyb prochází životním cyklem řízeným třemi metodami:

- vytvoritPohyb() – inicializuje nový pohyb: přiřadí ID, nastaví datum vzniku, typ pohybu a vazbu na součástku;
- overitData() – ověří platnost zadaných dat: existenci součástky, vyplnění povinných polí, dostatečné množství na skladě (příjem/výdej) nebo platnost lokací (přesun);
- provestPohyb() – fyzicky provede pohyb podle typu:
  - příjem → soucastka.zvysMnozstvi(pocet)
  - výdej → soucastka.snizMnozstvi(pocet)
  - přesun → zmenLokaci()

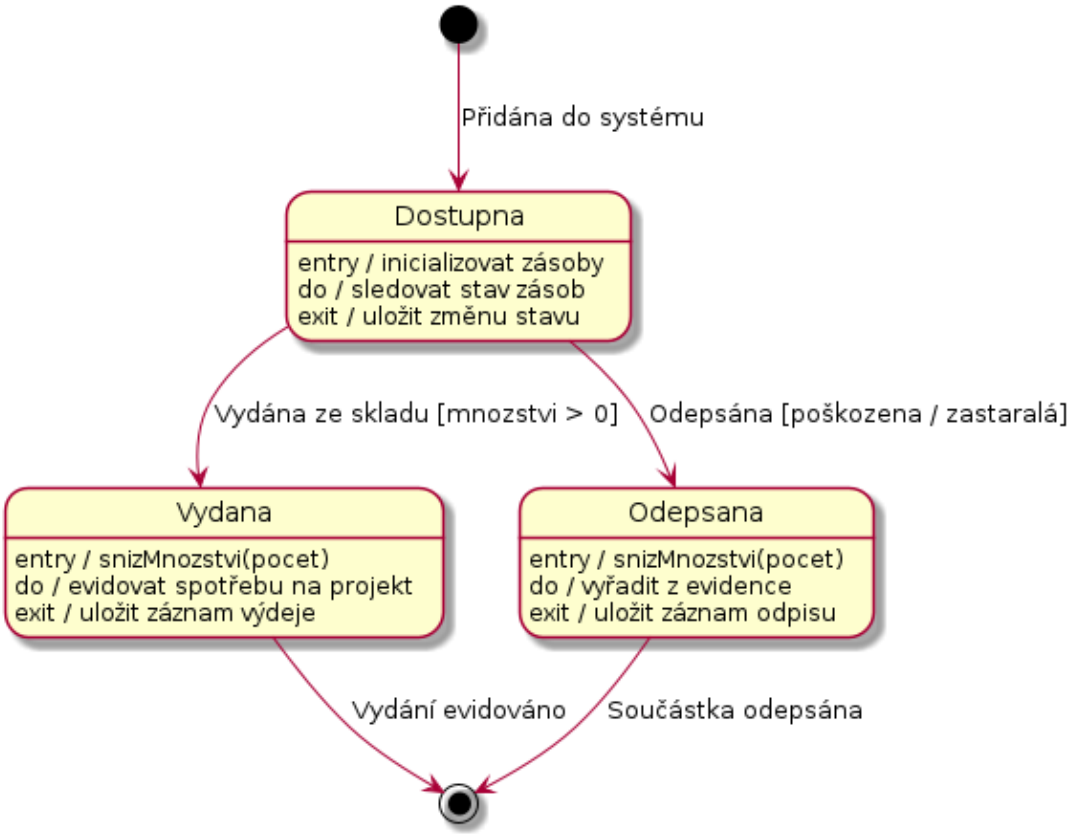


Obrázek 4.1: Stavový diagram třídy SkladovyPohyb

4.2 Stavový diagram třídy Soucastka

Diagram zachycuje, jak se mění stav součástky v průběhu skladových operací. Součástka je přidána do systému a okamžitě vstupuje do stavu Dostupna s nulovým množstvím. Stav Dostupna je hlavní pracovní stav součástky – může mít mnozstvi = 0 (čeká na naskladnění) i nenulové množství, přičemž část může být blokována atributem rezervovaneMnozstvi. Metoda overDostupnost() ověří, zda je volné množství (mnozstvi - rezervovaneMnozstvi > 0). Stav součástky je řízen těmito metodami:

- zvysMnozstvi(pocet) – zavolá se při naskladnění; zvýší atribut mnozstvi, součástka zůstává ve stavu Dostupna.
- rezervovat(pocet) – zablokuje část kusů pro projekt; zvýší rezervovaneMnozstvi, stav se nemění.
- zrusitRezervaci(pocet) – uvolní rezervaci; sníží rezervovaneMnozstvi, stav se nemění.
- snizMnozstvi(pocet) – zavolá se při výdeji ze skladu (Vydana) nebo odpisu (Odepsana); oba jsou terminální stavy.

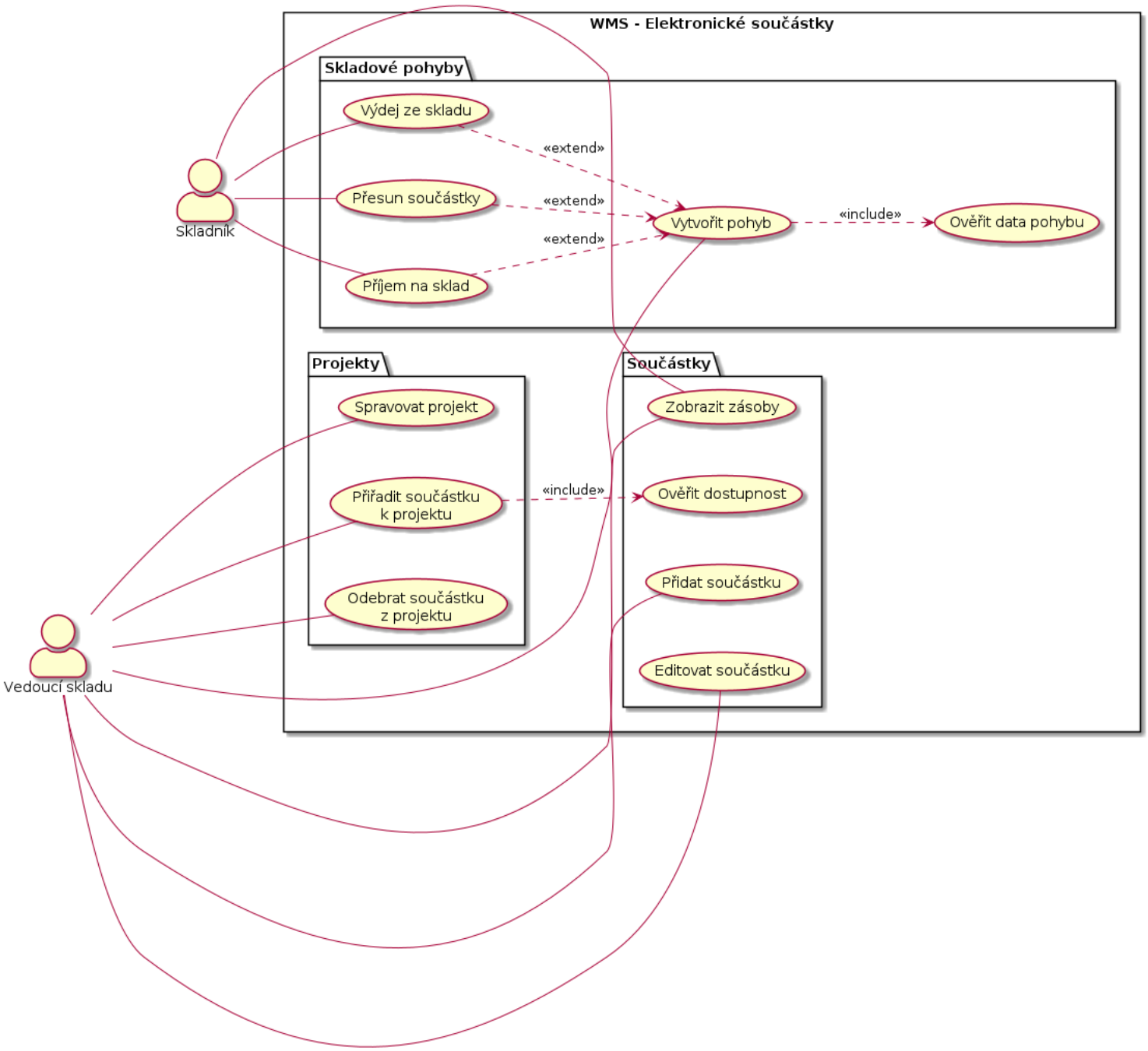


Obrázek 4.2: Stavový diagram třídy Soucastka

5. Model interakcí

5.1 Use Case diagram

Systém má dva hlavní aktéry. **Vedoucí skladu** spravuje součástky, projekty a vytváří skladové pohyby. **Skladník** provádí fyzické operace – příjem, výdej a přesun součástek a zobrazuje zásoby.



Obrázek 5.1: Use Case diagram

5.2 Scénáře

5.2.1 Scénář: Příjem na sklad (PrijemPohyb)

1. Uživatel zvolí typ pohybu *příjem* a zadá součástku, množství a dodavatele.
2. Systém volá overitData() – ověří existenci součástky a platnost cílové lokace.
3. Pokud data OK → systém volá provestPohyb(), který interně volá Soucastka.zvysMnozstvi().
4. Pohyb je potvrzen a zásoby jsou aktualizovány.
5. **Alternativní tok:** součástka neexistuje nebo lokace je neplatná → systém vrátí chybovou hlášku a pohyb je zamítnut.

5.2.2 Scénář: Výdej ze skladu (VydejPohyb)

1. Uživatel zvolí typ pohybu *výdej* a zadá součástku, množství a důvod výdeje.
2. Systém volá overitData() – ověří existenci součástky a zda mnozstvi $\geq$ požadovanému množství.
3. Pokud data OK → systém volá provestPohyb(), který interně volá Soucastka.snizMnozstvi().
4. Pohyb je potvrzen a zásoby jsou sníženy.
5. **Alternativní tok:** nedostatečné množství na skladě → systém vrátí chybovou hlášku a pohyb je zamítnut.

5.2.3 Scénář: Přesun součástky (PresunutiPohyb)

1. Uživatel zvolí typ pohybu *přesun* a zadá součástku, množství, zdrojovou a cílovou lokaci.
2. Systém volá overitData() – ověří existenci součástky, dostatečné množství ve zdrojové lokaci a platnost cílové lokace.
3. Pokud data OK → systém volá provestPohyb() a poté PresunutiPohyb.zmenLokaci().
4. Pohyb je potvrzen; součástka je evidována v cílové lokaci.
5. **Alternativní tok:** zdrojová nebo cílová lokace je neplatná → systém vrátí chybovou hlášku a pohyb je zamítnut.

5.2.4 Scénář: Přiřazení součástky k projektu

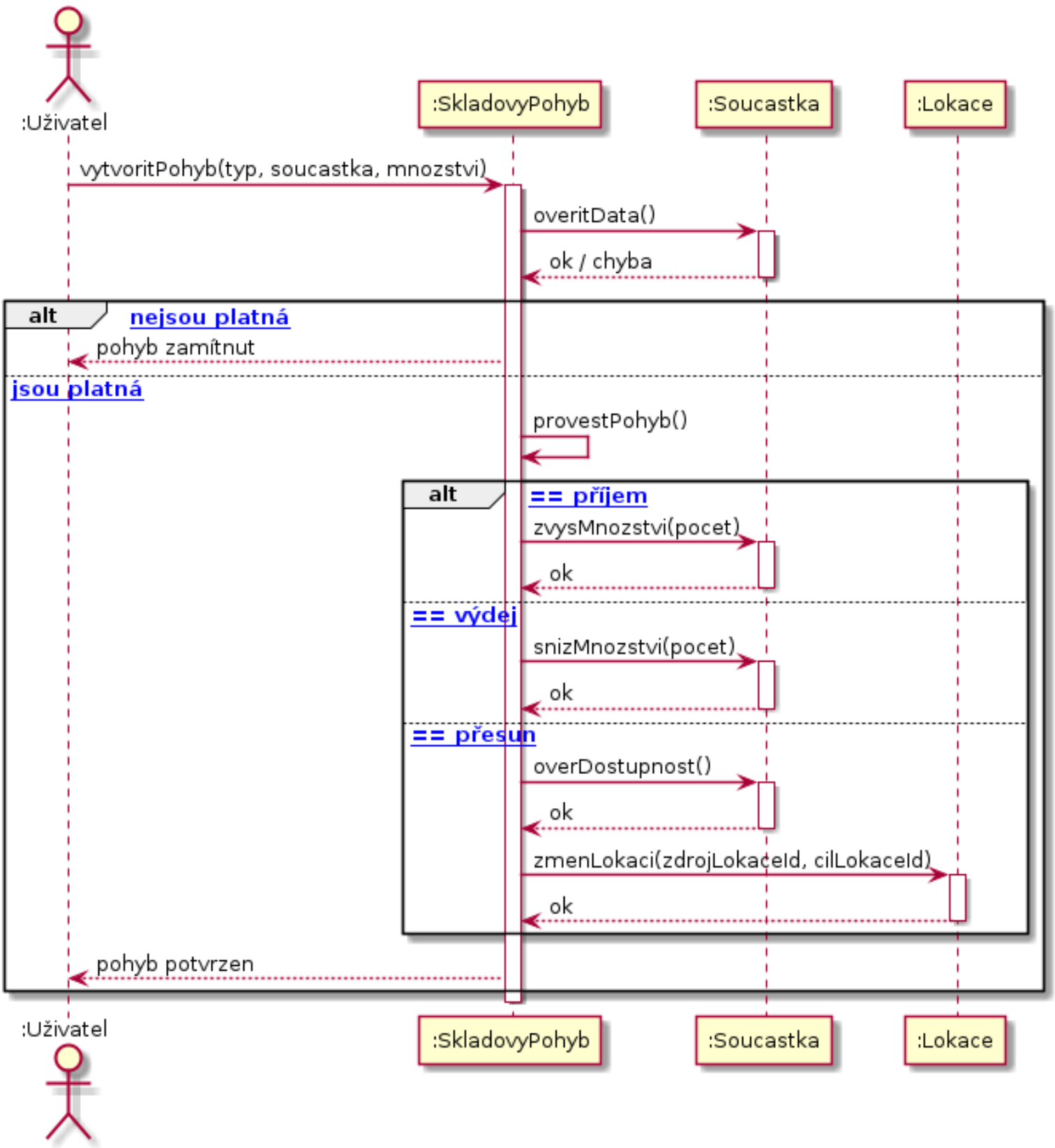
1. Uživatel vybere projekt a zadá součástku s množstvím.
2. Systém volá Soucastka.overDostupnost() – ověří, zda mnozstvi − rezervovaneMnozstvi > 0.
3. Pokud je dostatek volných kusů → systém volá Projekt.priradSoucastku(), které interně zavolá Soucastka.rezervovat().
4. Součástka je přiřazena k projektu a rezervované množství je aktualizováno.
5. **Alternativní tok:** nedostatečný počet volných kusů → systém vrátí chybovou hlášku a přiřazení je zamítnuto.

5.2.5 Scénář: Správa součástek

1. Uživatel přidá novou součástku nebo upraví existující.
2. Systém ověří validitu zadaných dat (název, počáteční množství, lokace).
3. Pokud data OK → součástka je uložena; zásoby jsou aktualizovány.
4. **Alternativní tok:** neplatná data → systém vrátí chybovou hlášku, součástka není uložena.

5.3 Sekvenční diagram – Vytvoření pohybu

Diagram zachycuje spolupráci objektů při vytvoření skladového pohybu. Uživatel volá vytvoritPohyb() na objektu :SkladovyPohyb, který ověří data voláním overitData() na objektu :Soucastka. Pokud data nejsou platná, pohyb je zamítnut. Pokud jsou data platná, pohyb je proveden voláním provestPohyb() – dle typu pohybu se volá příslušná metoda na :Soucastka nebo :Lokace. Po úspěšném provedení je pohyb potvrzen uživateli zprávou pohyb potvrzen.

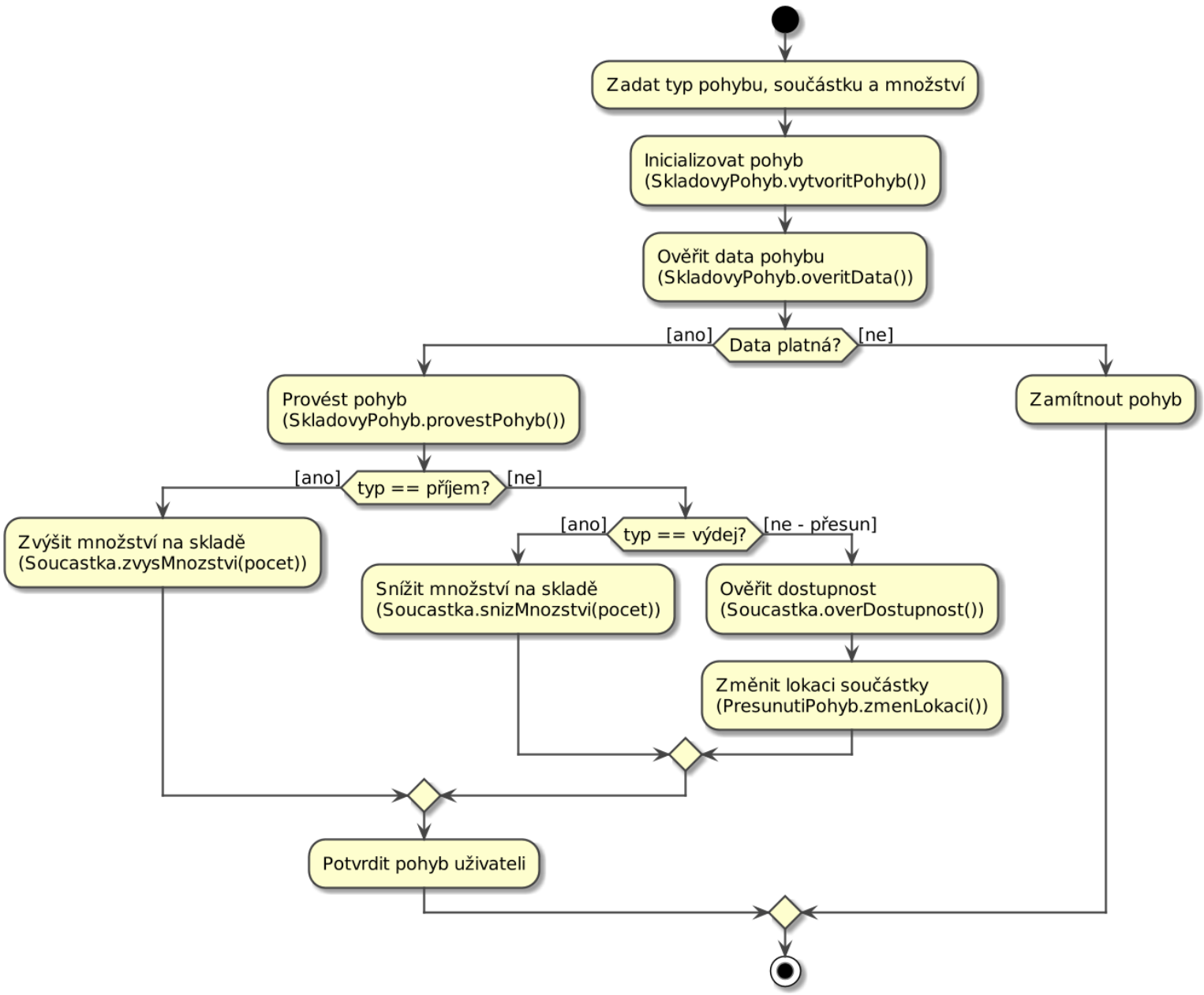


Obrázek 5.2: Sekvenční diagram – Vytvoření pohybu

5.4 Diagram aktivit – Vytvoření pohybu

Diagram zachycuje průběh vytvoření skladového pohybu v systému. Pohyb prochází třemi fázemi řízenými metodami třídy SkladovyPohyb:

- vytvoritPohyb() – uživatel zadá typ pohybu, součástku a množství; metoda inicializuje pohyb, přiřadí ID a datum vzniku.
- overitData() – ověří platnost zadaných dat: existenci součástky, dostatečné množství na skladě a platnost lokací u přesunu.
- provestPohyb() – fyzicky provede pohyb podle typu: příjem zavolá zvysMnozstvi(pocet), výdej snizMnozstvi(pocet), přesun zmenLokaci(). Pokud data nejsou platná, pohyb je zamítnut a uložena chyba.



Obrázek 5.3: Diagram aktivit – Vytvoření pohybu

6. Závěr

Cílem projektu bylo navrhnout objektový model informačního systému pro správu skladu elektronických součástek pomocí metod objektového, dynamického a funkčního modelování v jazyce UML.

Objektový model (class diagram) zachycuje strukturu systému – třídy, jejich atributy, metody a vzájemné vztahy. Diagram uplatňuje generalizaci (abstraktní třída SkladovyPohyb a její podtřídy), kvalifikovanou vazbu (přístup ke Soucastka přes kvalifikátor id: int), agregaci (Sklad obsahuje Lokace, Lokace ukládá Soucastky) a atribut spojení (ProjektSoucastka s atributem mnozstvi na vazbě mezi Projekt a Soucastka).

Dynamický model (stavové diagramy) popisuje chování dvou klíčových tříd. Stavový diagram třídy SkladovyPohyb zachycuje životní cyklus pohybu od vytvoření přes validaci až po provedení nebo zamítnutí. Stavový diagram třídy Soucastka popisuje přechody mezi stavy Dostupna, Vydana a Odepsana v závislosti na skladových operacích. Oba diagramy využívají akce entry, do a exit uvnitř stavů.

Model interakcí zachycuje komunikaci mezi aktéry a systémem. Use Case diagram definuje případy užití a jejich vztahy («include», «extend») pro dva aktéry – Vedoucího skladu a Skladníka. Scénáře popisují průběh klíčových operací včetně alternativních toků. Sekvenční diagram ukazuje spolupráci objektů při vytvoření skladového pohybu se synchronními zprávami a alternativními větvemi (alt). Diagram aktivit zachycuje tok řízení s podmínkovými větvemi a popisky v hranatých závorkách.

Jednotlivé modely se navzájem doplňují – objektový model definuje třídy, na které odkazují stavové diagramy i sekvenční diagram. Scénáře popisují stejné operace, které jsou graficky zachyceny v sekvenčním diagramu a diagramu aktivit.

Projekt pokrývá základní část systému – správu součástek, skladových pohybů a projektů. Jako možné rozšíření by bylo vhodné doplnit správu dodavatelů a nákupních objednávek, modul pro inventuru skladu a systém automatických notifikací při podkročení minimálního stavu zásob.

A. Zdrojové kódy PlantUML diagramů

Tato příloha obsahuje zdrojové kódy všech UML diagramů vygenerovaných nástrojem PlantUML.

A.1 Use Case diagram

```
@startuml 01_use_case
left to right direction
skinparam actorStyle awesome
skinparam nodesep 50
skinparam ranksep 60
skinparam packagePadding 10

actor "Vedoucí skladu" as Vedouci
actor "Skladník" as Skladnik

Vedouci -[hidden]down- Skladnik

rectangle "WMS     Elektronické součástky" {

  package "Projekty" {
    usecase "Spravovat projekt" as UC_PROJEKT
    usecase "Přiřadit součástku\nk projektu" as UC_PRIRADIT
    usecase "Odebrat součástku\nz projektu" as UC_ODEBRAT
  }

  package "Součástky" {
    usecase "Přidat součástku" as UC_ADD
    usecase "Editovat součástku" as UC_EDIT
    usecase "Zobrazit zásoby" as UC_ZASOBY
    usecase "Ověřit dostupnost" as UC_DOSTUPNOST
  }

  package "Skladové pohyby" {
    usecase "Příjem na sklad" as UC_PRIJEM
    usecase "V dej ze skladu" as UC_VYDEJ
    usecase "Přesun součástky" as UC_PRESUN
    usecase "Vytvořit pohyb" as UC_POHYB
    usecase "Ověřit data pohybu" as UC_OVERIT
  }

}

' Vedoucí skladu     nahoře
Vedouci -- UC_PROJEKT
Vedouci -- UC_PRIRADIT
Vedouci -- UC_ODEBRAT
Vedouci -- UC_ADD
Vedouci -- UC_EDIT
Vedouci -- UC_ZASOBY
Vedouci -- UC_POHYB

' Skladník     dole
Skladnik -- UC_PRIJEM
Skladnik -- UC_VYDEJ
Skladnik -- UC_PRESUN
Skladnik -- UC_ZASOBY

' include
UC_POHYB    ..> UC_OVERIT     : <<include>>
UC_PRIRADIT ..> UC_DOSTUPNOST : <<include>>

' extend
UC_PRIJEM ..> UC_POHYB : <<extend>>
UC_VYDEJ  ..> UC_POHYB : <<extend>>
UC_PRESUN ..> UC_POHYB : <<extend>>

@enduml
```

A.2 Class diagram

```
@startuml
top to bottom direction
skinparam CircledCharacterRadius 0
skinparam CircledCharacterFontSize 0
skinparam defaultFontSize 13
skinparam classFontSize 14
skinparam classFontStyle bold
skinparam ArrowFontSize 12
skinparam nodesep 50
skinparam ranksep 60

class Uzivatel {
  id: int
  jmeno: String
  email: String
}

class PrijemPohyb {
  dodavatel: String
  vytvoritPohyb(): void
}

class VydejPohyb {
  duvod: String
  vytvoritPohyb(): void
}

class PresunutiPohyb {
  zdrojLokaceId: int
  cilLokaceId: int
  vytvoritPohyb(): void
  zmenLokaci(): void
}

abstract class SkladovyPohyb {
  id: int
  datum: Date
  stav: String
  typ: String
  mnozstvi: int
  {abstract} vytvoritPohyb(): void
  overitData(): boolean
  provestPohyb(): void
}

class Projekt {
  id: int
  nazev: String
  priradSoucastku(Soucastka): void
  odeberSoucastku(Soucastka): void
}

class ProjektSoucastka {
  mnozstvi: int
}

class Sklad {
  id: int
  nazev: String
  najdiSoucastku(id: int): Soucastka
}

class Lokace {
  id: int
  oznaceni: String
}

class Soucastka {
  id: int
  nazev: String
  mnozstvi: int
  rezervovaneMnozstvi: int
  stav: String
  zvysMnozstvi(pocet: int): void
  snizMnozstvi(pocet: int): void
  rezervovat(pocet: int): void
```

```
  zrusitRezervaci(pocet: int): void
  overDostupnost(): boolean
}

'        Generalizace


PrijemPohyb    --|> SkladovyPohyb
VydejPohyb     --|> SkladovyPohyb
PresunutiPohyb --|> SkladovyPohyb

'        Atribut spojení


(Projekt, Soucastka) .. ProjektSoucastka
Projekt -[hidden]left- ProjektSoucastka

'        Kvalifikovaná vazba


Sklad "[id: int]" --> "1" Soucastka : najdiSoucastku

'        Asociace


SkladovyPohyb "*" --> "1" Soucastka     : je součástí
Sklad         "1" *-- "*" Lokace        : obsahuje
Lokace        "1" o-- "*" Soucastka     : ukládá
SkladovyPohyb "*" --> "1" Sklad         : mění stav
Uzivatel      "1" --> "*" SkladovyPohyb : vytváří
Uzivatel      "1" --> "*" Projekt       : spravuje

'        Layout


Uzivatel    -[hidden]right- PrijemPohyb
PrijemPohyb -[hidden]right- VydejPohyb
VydejPohyb  -[hidden]right- PresunutiPohyb

@enduml
```

A.3 Stavový diagram – SkladovyPohyb

```
@startuml
top to bottom direction

[*] --> Vytvoren : / vytvoritPohyb()

Vytvoren : entry / přiřadit ID a datum
Vytvoren : do / čekat na odeslání
Vytvoren : exit / uložit pohyb
Vytvoren --> Validovan : Pohyb odeslán

Validovan : entry / overitData()
Validovan : do / kontrolovat data
Validovan : exit / potvrdit v sledek
Validovan --> Proveden_Prijem : Pohyb schválen [typ == příjem]
Validovan --> Proveden_Vydej  : Pohyb schválen [typ == v dej]
Validovan --> Proveden_Presun : Pohyb schválen [typ == přesun]
Validovan --> Zamitnut        : Chyba ověření [neplatná data]
Validovan --> Zamitnut        : Chyba ověření [nedostatečné množství]

Zamitnut : entry / zaznamenat chybu
Zamitnut : do / čekat na notifikaci
Zamitnut : exit / notifikovat uživatele

Proveden_Prijem : entry / soucastka.zvysMnozstvi()
Proveden_Prijem : do / aktualizovat zásoby
Proveden_Prijem : exit / uložit záznam

Proveden_Vydej : entry / soucastka.snizMnozstvi()
Proveden_Vydej : do / aktualizovat zásoby
Proveden_Vydej : exit / uložit záznam

Proveden_Presun : entry / zmenLokaci()
Proveden_Presun : do / aktualizovat lokaci
Proveden_Presun : exit / uložit záznam

Proveden_Prijem --> [*] : Pohyb dokončen
Proveden_Vydej  --> [*] : Pohyb dokončen
Proveden_Presun --> [*] : Pohyb dokončen
Zamitnut        --> [*] : Pohyb zamítnut

@enduml
```

A.4 Stavový diagram – Soucastka

```
@startuml
top to bottom direction

[*] --> Dostupna : Přidána do systému

Dostupna : entry / inicializovat zásoby
Dostupna : do / sledovat stav zásob
Dostupna : exit / uložit změnu stavu
Dostupna --> Vydana   : Vydána ze skladu [mnozstvi > 0]
Dostupna --> Odepsana : Odepsána [poškozena / zastaralá]

Vydana : entry / snizMnozstvi(pocet)
Vydana : do / evidovat spotřebu na projekt
Vydana : exit / uložit záznam v deje
Vydana --> [*] : Vydání evidováno

Odepsana : entry / snizMnozstvi(pocet)
Odepsana : do / vyřadit z evidence
Odepsana : exit / uložit záznam odpisu
Odepsana --> [*] : Součástka odepsána

@enduml
```

A.5 Sekvenční diagram – Vytvoření pohybu

```
@startuml
skinparam defaultFontSize 13
skinparam sequenceArrowThickness 1.5
skinparam sequenceParticipantBorderThickness 1.5
skinparam sequenceGroupBorderThickness 1.5

' ->    plná čára, plná šipka    = synchronní zpráva (odesílatel čeká)
' -->   přerušovaná čára         = odpově   / návratová hodnota
' ->>   plná čára, prázdná šipka = asynchronní zpráva (odesílatel NEČEKÁ)

actor ":Uživatel" as Uzivatel
participant ":SkladovyPohyb" as Pohyb
participant ":Soucastka" as Soucastka
participant ":Lokace" as Lokace

Uzivatel -> Pohyb : vytvoritPohyb(typ, soucastka, mnozstvi)
activate Pohyb

Pohyb -> Soucastka : overitData()
activate Soucastka
Soucastka --> Pohyb : ok / chyba
deactivate Soucastka

alt [data nejsou platná]
    Pohyb --> Uzivatel : pohyb zamítnut

else [data jsou platná]
    Pohyb -> Pohyb : provestPohyb()

    alt [typ == příjem]
        Pohyb -> Soucastka : zvysMnozstvi(pocet)
        activate Soucastka
        Soucastka --> Pohyb : ok
        deactivate Soucastka

    else [typ == v dej]
        Pohyb -> Soucastka : snizMnozstvi(pocet)
        activate Soucastka
        Soucastka --> Pohyb : ok
        deactivate Soucastka

    else [typ == přesun]
        Pohyb -> Soucastka : overDostupnost()
        activate Soucastka
        Soucastka --> Pohyb : ok
        deactivate Soucastka
        Pohyb -> Lokace : zmenLokaci(zdrojLokaceId, cilLokaceId)
        activate Lokace
        Lokace --> Pohyb : ok
        deactivate Lokace
    end

    Pohyb --> Uzivatel : pohyb potvrzen
end

deactivate Pohyb

@enduml
```

A.6 Diagram aktivit – Vytvoření pohybu

```
@startuml
scale 1.5
skinparam defaultFontSize 13
skinparam ArrowThickness 1.5
skinparam ActivityBorderThickness 1.5
skinparam ActivityDiamondBorderThickness 1.5
skinparam ActivityBorderColor #444444
skinparam ActivityDiamondBorderColor #444444
skinparam ArrowColor #444444

start

:Zadat typ pohybu, součástku a množství;
:Inicializovat pohyb\n(SkladovyPohyb.vytvoritPohyb());
:Ověřit data pohybu\n(SkladovyPohyb.overitData());

if (Data platná?) then ([ano])
  :Provést pohyb\n(SkladovyPohyb.provestPohyb());

  if (typ == příjem?) then ([ano])
    :Zv šit množství na skladě\n(Soucastka.zvysMnozstvi(pocet));
  else ([ne])
    if (typ == v dej?) then ([ano])
      :Snížit množství na skladě\n(Soucastka.snizMnozstvi(pocet));
    else ([ne     přesun])
      :Ověřit dostupnost\n(Soucastka.overDostupnost());
      :Změnit lokaci součástky\n(PresunutiPohyb.zmenLokaci());
    endif
  endif

  :Potvrdit pohyb uživateli;

else ([ne])
  :Zamítnout pohyb;
endif

stop
@enduml
```